Inductive Bias Meta-Learning with Generative Models

Anna Novokshonova
MIPT
Moscow, Russia
email@phystech.edu

Oleg Bakhteev
MIPT
Moscow, Russia
email@phystech.edu

Fedor Sobolevsky
MIPT
Moscow, Russia
email@phystech.edu

Muhammadsharif Nabiev
MIPT
Moscow, Russia
email@phystech.edu

Abstract

This paper investigates inductive bias in machine learning models. By inductive bias we mean the preference of a model for certain types of functions or data structures over others. To analyze the inductive bias of a fixed model, we consider a problem of finding data that this model can fit and generalize on particularly well. Previous work demonstrated that generating labels for a fixed dataset allows one to extract the inductive bias. Here, we extend this approach by proposing a method for generating full synthetic datasets. We train a generative model to produce datasets on which the target model achieves strong generalization performance. We test the proposed framework on CNNs and RNNs, and analyze obtained datasets for each model.

Keywords: inductive bias, meta-learning, generative models, CNN, RNN.

1 Introduction

Inductive bias plays a fundamental role in a model's ability to generalize [1]. Without additional assumptions, any hypothesis that fits a finite training dataset is considered equally likely by the model, which can lead to poor generalization. Thus, effective generalization requires a set of prior assumptions about the data, and these assumptions constitute the model's inductive bias. Understanding and exploiting inductive bias can be a powerful tool for enhancing model performance.

The term "inductive bias" does not have a single, strict definition. For instance, inductive bias can mean underlying assumptions about the domain and task or specific implementation choices that constrain or order the space of hypotheses [2]. In this paper, by inductive bias we mean a model's preference for certain types of functions or data structures over others, expressed through better generalization performance. For example, convolutional neural networks (CNNs) are inductively biased towards data with local structure and translation invariance [3–5], while recurrent neural networks (RNNs) are inductively biased towards ordered data with sequential structure [6–8].

A clear understanding of the correspondence between models (or their specific components) and inductive biases is useful for solving machine learning problems. However, this task is highly non-trivial: for most models with complex architecture, the problem of detecting inductive bias is analytically intractable. Several approaches have been developed to extract and characterize the inductive bias of models. Boopathy et al. [9] proposed a method to quantify the amount of inductive bias of tasks or models as the amount of information required to specify well-generalizing models within a given hypothesis space. Si et al. [10] investigated the inductive bias of in-context learning

39th Conference on Neural Information Processing Systems (NeurIPS 2025).

by constructing underspecified demonstrations. Li et al. [11] proposed a framework to capture the inductive biases in a learning system by meta-learning Gaussian process kernel hyperparameters from its predictions.

One recent result in this area is the meta-learning framework introduced by Dorrell et al. [12], which identifies the inductive bias of differentiable supervised learning algorithms by generating labels for a fixed dataset. For all its strengths, this approach has certain limitations. First, it requires access to input data or its distribution. Second, its analysis of inductive bias is constrained by the fixed dataset and its structure.

Here, we propose to extend this approach by generating entire synthetic datasets. The meta-learning loop proceeds as follows: an outer generative model, or meta-learner, produces a dataset, which is used to train and test an inner model, or learner, whose inductive bias we aim to extract. After the learner is trained, the quality of its predictions on test data from this dataset is incorporated into the meta-learner's loss function. Thus, the meta-learner learns to generate datasets on which the target model shows the best generalization performance; these are datasets towards which it is inductively biased.

The proposed approach requires no knowledge of the input data distribution, making it more flexible. We expect it to enable a more comprehensive analysis of inductive bias.

Contributions. Our contributions can be summarized as follows:

- We present a generative meta-learning framework for extracting the inductive bias of a target model. Unlike prior work [12] that generates only labels for samples from a fixed distribution, our approach produces entire synthetic datasets via a meta-learned generative model.
- We empirically evaluate our algorithm on CNNs and RNNs and compare the results with the known inductive biases of these architectures. For CNNs, we expect to detect a local structure in the generated datasets; for RNNs, we expect them to have a sequential structure.
- We highlight the implications of our findings for understanding and leveraging inductive bias. Our method is general-purpose: it can be used to extract inductive bias of any differentiable supervised learning algorithm and, consequently, to improve the generalization performance of models in machine learning problems.

Outline. The rest of the paper is organized as follows. First, we formally state the problem we address and describe the proposed method in detail. Next, we validate it through experiments with CNNs and RNNs, analyzing the obtained datasets for the well known inductive biases of these architectures. Finally, we discuss the results and conclude the paper.

2 Problem statement

We study the problem of extracting and characterizing the inductive bias of learning models by finding the datasets that they generalize best. We now present a formal statement of the problem.

Let $\mathbb{X} \subset \mathbb{R}^n$ be the input space, $\mathbb{Y} \subset \mathbb{R}^m$ be the output space and $\mathbf{f}_{\boldsymbol{\theta}} : \mathbb{X} \to \mathbb{Y}$ be the target model, where $\boldsymbol{\theta} \in \Theta \subset \mathbb{R}^k$ are its parameters. Let $\ell(\mathbf{f}_{\boldsymbol{\theta}}(\mathbf{x}), \mathbf{y})$ be the loss function of the target model.

Consider a parametric family of distributions $p(\mathbf{x}|\mathbf{y}, \mathbf{w})$, where $\mathbf{w} \in \mathbb{W} \subset \mathbb{R}^l$ are the parameters of the distribution. For a fixed label $\mathbf{y} \in \mathbb{Y}$, a sample $\mathbf{x} \sim p(\mathbf{x}|\mathbf{y}, \mathbf{w})$ is produced by the generator $\mathbf{g}(\mathbf{w})$.

We aim to find data distribution function p^*, corresponding to parameters $\mathbf{w}^*$, the solution to the following optimization problem:

$$\mathbf{w}^* = \arg\min_{\mathbf{w} \in \mathbb{W}} \mathrm{E}_{\mathfrak{D}_{\text{syn}} \sim p(\mathbf{x}|\mathbf{y},\mathbf{w})} \left[\mathcal{L}\left(\mathbf{f}_{\hat{\boldsymbol{\theta}}}, \mathfrak{D}_{\text{test}}\right) \right] \quad \text{s.t.} \quad \mathfrak{D}_{\text{syn}} = \mathfrak{D}_{\text{train}} \cup \mathfrak{D}_{\text{test}} \in \mathcal{U}$$

where

$$\hat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta} \in \Theta} \sum_{(\mathbf{x},\mathbf{y}) \in \mathfrak{D}_{\text{train}}} \ell(\mathbf{f}_{\boldsymbol{\theta}}(\mathbf{x}), \mathbf{y}).$$

$\mathfrak{D}_{\text{syn}}$ denotes the synthetic dataset generated by $\mathbf{g}(\mathbf{w})$. In the optimization problem, the minimum is sought not over all possible datasets, but only over those belonging to a set $\mathcal{U}$, which may vary

depending on the problem setting. In our case, $\mathcal{U}$ defines the structure of the dataset: the number of classes is specified, and for each class, $\mathfrak{D}_{\text{syn}}$ contains a fixed number of samples, which is the same for every class.

Function $\mathcal{L}$ measures a model's ability to generalize by calculating the prediction error on a test set and also contains regularization terms to avoid trivial solutions. For example, prediction error can be estimated as cross-entropy or mean squared error (MSE), defined respectively as:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{|\mathfrak{D}_{\text{test}}|} \sum_{(\mathbf{x},\mathbf{y})\in\mathfrak{D}_{\text{test}}} \sum_{i=1}^{m} y_i \log\left([\mathbf{f}_{\hat{\boldsymbol{\theta}}}(\mathbf{x})]_i\right), \qquad \mathcal{L}_{\text{MSE}} = \frac{1}{|\mathfrak{D}_{\text{test}}|} \sum_{(\mathbf{x},\mathbf{y})\in\mathfrak{D}_{\text{test}}} \|\mathbf{y} - \mathbf{f}_{\hat{\boldsymbol{\theta}}}(\mathbf{x})\|^2.$$

3 Method

Our proposed approach is to meta-train a generative model to produce datasets on which the target model exhibits the best generalization performance, i.e., datasets to which it is inductively biased. In this section we describe our method in detail.

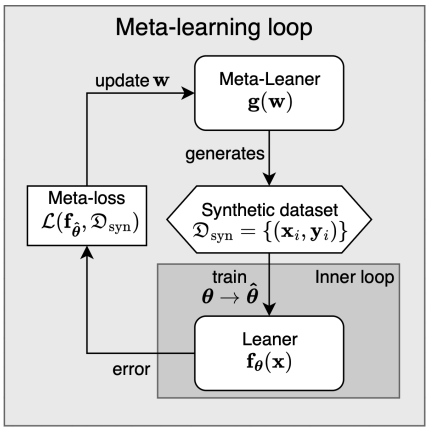


Figure 1: Meta-learning scheme

The meta-learning cycle is structured as follows. First, the generator $\mathbf{g}(\mathbf{w})$ with current parameters $\mathbf{w}$ constructs a synthetic dataset $\mathfrak{D}_{\text{syn}}$ according to the predefined configuration: for each class, a fixed number of samples is generated. The number of classes and the number of samples per class are both specified as hyperparameters. Then, the resulting dataset is split into training $\mathfrak{D}_{\text{train}}$ and test $\mathfrak{D}_{\text{test}}$ sets.

Next, the target model $\mathbf{f}_{\boldsymbol{\theta}}$ is trained on the training set $\mathfrak{D}_{\text{train}}$ in the inner loop. It is important that the model be re-initialized from scratch at each meta-learning iteration. At the output of the inner loop we obtain a trained model $\mathbf{f}_{\hat{\boldsymbol{\theta}}}$.

Next, the meta-loss function is calculated. It consists of two terms. The first term $\mathcal{L}_g$ measures the model's generalization quality, estimated on the test set $\mathfrak{D}_{\text{test}}$. This could be, for example, MSE or Cross-Entropy. The second term $\mathcal{R}$ is the regularization term. Note that if all objects within a class are identical, the target model would easily achieve 100% accuracy. However, this degenerate case is uninformative. The regularization term is precisely what prevents such trivial solutions. Possible choices include the average distance between objects within the same class, defined as:

$$\mathcal{R}_{\text{dist}} = -\frac{1}{C} \sum_{k=1}^{C} \frac{1}{n_k(n_k-1)} \sum_{\substack{i,j\in\mathcal{I}_k \\ i\neq j}} \|\mathbf{x}_i - \mathbf{x}_j\|_2^2,$$

where C is the number of classes in the dataset, n_k is the number of objects in class k, and $\mathcal{I}_k$ is the set of indices of objects belonging to class k.

Another regularization strategy is to penalize small singular values of the generator's weight matrices, as this helps to avoid degeneracy. The corresponding regularization term is defined as:

$$\mathcal{R}_{\text{svd}} = \frac{1}{K} \sum_{k=1}^{K} \frac{1}{r_k} \sum_{i=1}^{r_k} \left(\max(\tau - \sigma_i^{(k)}, 0)\right)^2,$$

where K is the number of linear layers in the generator, $\sigma_1^{(k)}, \ldots, \sigma_{r_k}^{(k)}$ are the singular values of the k-th weight matrix and τ is a fixed threshold.

Thus, during meta-training, the meta-loss is minimized by updating the generator parameters $\mathbf{w}$, resulting in a generator $\mathbf{g}(\hat{\mathbf{w}})$ that produces datasets which are easy for the target model to generalize from. The structure of these datasets reflects the model's inductive bias.

The proposed meta-learning procedure is summarized in Algorithm 1.

Algorithm 1 Meta-learning for inductive bias extraction

Initialize generator $\mathbf{g}(\mathbf{w})$ with random parameters $\mathbf{w}$
while step $<$ total steps **do**
    Generate synthetic dataset $\mathfrak{D}_{\text{syn}} = \{(\mathbf{x}_i, \mathbf{y}_i)\}$ with C classes and N samples per class
    Split $\mathfrak{D}_{\text{syn}}$ into training set $\mathfrak{D}_{\text{train}}$ and test set $\mathfrak{D}_{\text{test}}$
    Re-initialize target model $\mathbf{f}_{\boldsymbol{\theta}}$ from scratch
    Train $\mathbf{f}_{\boldsymbol{\theta}}$ on $\mathfrak{D}_{\text{train}}$ in the inner loop $\rightarrow$ trained model $\mathbf{f}_{\hat{\boldsymbol{\theta}}}$
    Compute meta-loss: $\mathcal{L}_{\text{meta}} = \mathcal{L}_g(\mathbf{f}_{\hat{\boldsymbol{\theta}}}, \mathfrak{D}_{\text{test}}) + \lambda \mathcal{R}(\mathfrak{D}_{\text{syn}})$
    Take $\mathbf{w}$ gradient step on meta-loss
end while

Similar to previous work [12], our approach can be extended to compare the inductive biases of two models, i.e., to find a dataset that is easy for one model to generalize from but difficult for the other. In this case, both models are trained in the inner loop, and the measure of the generalization quality of the second model is subtracted when computing the meta-loss. Consequently, minimizing this meta-loss improves the generalization of the first model while degrading that of the second.

Finally, to improve the stability of meta-training, we use several independently initialized learners at each meta-training iteration and average their contributions to the meta-loss. All learners are trained on the same train-test split of the generated dataset. If K learners are used, the meta-loss has the form:

$$\mathcal{L}_{\text{meta}} = \frac{1}{K} \sum_{k=1}^{K} \mathcal{L}_g \left(\mathbf{f}_{\hat{\boldsymbol{\theta}}_k}, \mathfrak{D}_{\text{test}} \right) + \lambda \mathcal{R}(\mathfrak{D}_{\text{syn}}),$$

where $\hat{\boldsymbol{\theta}}_k$ denotes the parameters of the k-th learner after the inner optimization. This averaging reduces variation in the meta-loss and its gradient caused by the random initialization of the target model.

4 Experiments

To empirically validate our algorithm, we conducted computational experiments to extract the inductive bias of CNNs and RNNs. Specifically, we verify that the generative model learns to create datasets on which the target model exhibits the best generalization performance and attempt to interpret the obtained results in light of the well-known inductive biases of these architectures.

All data in our experiments are fully synthetic. In all experiments, we use a trainable tensor as the generative model, meaning that the elements of the synthetic dataset are directly optimized as parameters. The structure of the datasets is defined by a given number of classes and a fixed number of samples per class.

The meta-learning process is implemented using the learn2learn library [13]. Specifically, to enable end-to-end differentiability, we wrap the inner learner in a MAML (Model-Agnostic Meta-Learning) wrapper [14]. This allows us to clone the learner for each meta-step, train it on the generated dataset, and still keep the computational graph needed to propagate gradients from the meta-loss back to the generator. For the inner loop optimization, we use SGD. For the outer loop (meta-optimization), we use Adam [15].

The source code for all experiments is available at GitHub[1].

4.1 CNNs and 1×4 Images

We first test our method on a simple example: CNNs with a 1×2 convolution and 1×4 images.

The CNN architecture is as follows: a convolution with a 1×2 kernel and stride (1,2), followed by max pooling, and finally a linear layer that outputs class logits. In the experiment we uses the following

[1] Code at `https://github.com/intsystems/2026-Project-205`

parameters: 2 classes, 20 samples per class, 7 inner loop steps, and learning rates of 0.5 and 0.05 for inner and outer loop respectively. As a measure of prediction error on the test set, we use MSE. First, we try without regularization. Then, we add regularization via the average intra-class distance with a parameter $\lambda = 0.5$.

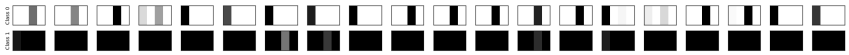


Figure 2: Obtained dataset without regularization



Figure 3: Obtained dataset for average intra-class distance regularization

As can be seen in Figure 2, even without regularization, the trivial solution — where all images within a class are identical — does not fully occur. This reveals the convolutional architecture's inductive bias towards locality: within a given class, we observe images where the first and second pairs of pixels are swapped. This indicates that for this architecture, local structure matters more than global structure.

Adding the mean intra-class distance as a regularization term makes the classes more diverse while preserving locality. The obtained dataset is presented in Figure 3. In class 1, the defining pattern is a white pixel followed by a black pixel (in that exact order) in one of the two positions, while the other position can contain anything — two black pixels, two gray, one white and one black (except for zero class patterns). Moreover, this pattern may appear either in the left or the right pair. This reflects the inductive bias towards locality: the label is determined by the local structure rather than the global one. The effect of stride (1,2) is also noticeable — class 0 contains an element with a white pixel followed by a black pixel in the middle, but this does not cause it to be misclassified as class 1.

To verify locality, we performed the following. We first split the obtained dataset into training and test sets. We then swapped the left and right parts of the images in the test set. After training the model on the original training set, we evaluated its accuracy on the swapped test set. Similarly, we trained the model on the swapped training set and evaluated it on the original test set. The results were averaged over 1000 random train-test splits. As can be seen from Table 1, the model achieves similar accuracy in both cases, confirming the locality of the data.

Table 1: Accuracy comparison on original and swapped data

Training	**Test**	**Accuracy (%)**
Original	Original	98.78 ± 8.82
Original	Swapped	98.78 ± 8.82
Swapped	Swapped	99.08 ± 7.25
Swapped	Original	99.08 ± 7.25

Thus, the structure of the obtained datasets reflects the inductive bias of CNNs toward data with local structure, which is consistent with previously known results.

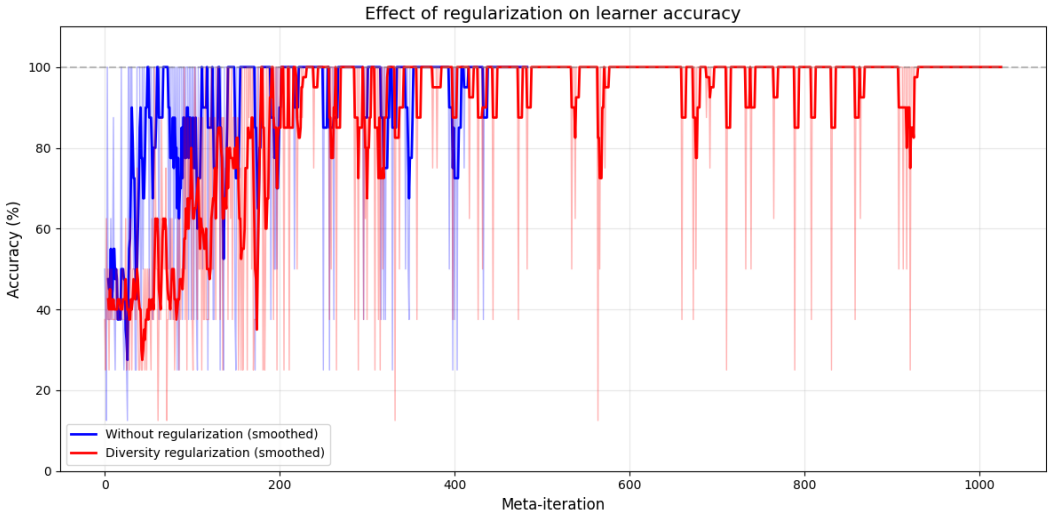


Figure 4: Comparison of meta-loss dynamics with and without regularization

4.2 CNNs and 4x4 images

Next, we conduct an experiment with CNNs on 4×4 images.

First, we consider the following CNN architecture. It consists of a single convolutional layer with a 2×2 kernel and stride 2, which reduces a 4×4 input to 4 feature maps of size 2×2. This is followed by ReLU activation and 2×2 max pooling, resulting in 4 feature maps of size 1×1. After flattening, a fully connected layer maps the 4 features to the output classes.

The experimental parameters are as follows: 4 classes, 20 samples per class, 10 inner loop steps, and learning rates of 0.5 and 0.01 for the inner and outer loops, respectively.

As meta-loss functions, we experiment with both MSE and cross-entropy (denoted CE). For regularization, we consider two strategies: the average intra-class distance (denoted Dist) and a penalty on small singular values of the generator's weight matrices (denoted SVD). We evaluate all four combinations of meta-loss and regularization. The selected regularization parameter λ for each configuration are summarized in Table 2.

Table 2: λ for different meta-loss combinations

	Dist	SVD
CE	1.5	1.2
MSE	0.1	0.5

The obtained datasets are presented on Figures 5-8.

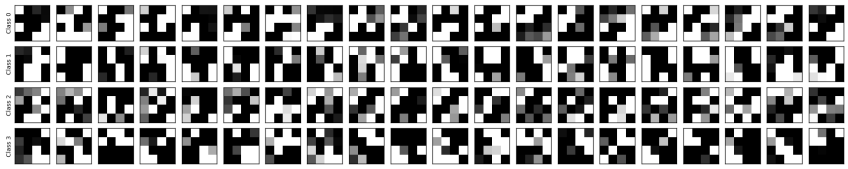


Figure 5: Obtained dataset for cross-entropy and average intra-class distance regularization

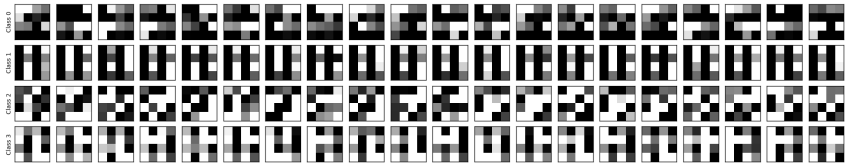


Figure 6: Obtained dataset for cross-entropy and SVD regularization

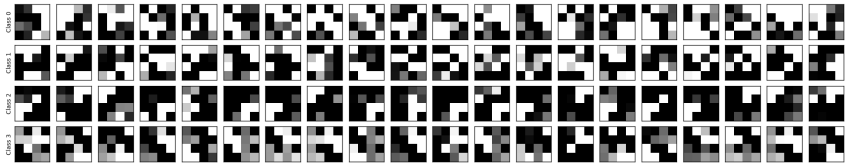


Figure 7: Obtained dataset for MSE and average intra-class distance regularization

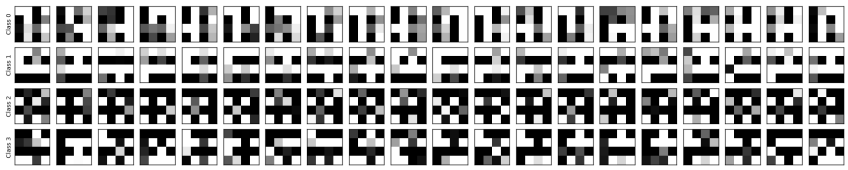


Figure 8: Obtained dataset for MSE and SVD regularization

Figure 9 compares the accuracy dependence on meta-iteration across different configurations.

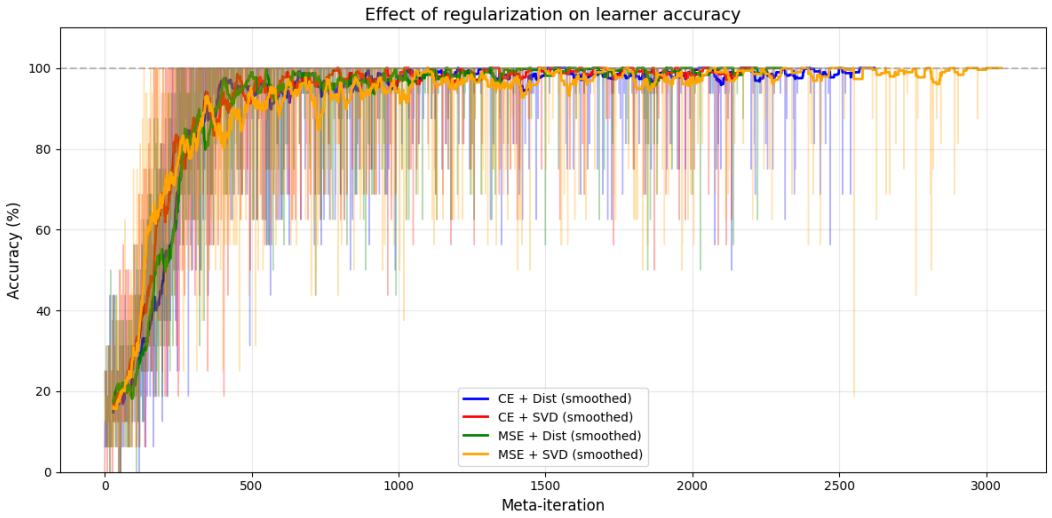


Figure 9: Comparison on different meta-loss strategies

To characterize the properties of the obtained datasets, we carry out additional analysis to verify their locality. For each configuration of the meta-loss function, we train the target model on the original training set and then evaluate its accuracy on both the original test set and a shuffled version of the test set, where the 2×2 blocks within each image are randomly permuted. The results are averaged over 1000 random train-test splits and over different seeds for shuffling. All results are summarized in Table 3.

Next, we compare our CNN with an MLP, a model that has no explicit inductive bias toward locality and is sensitive to global structure. We repeat the experiment using the same inner-loop parameters:

training on the original training set and evaluating on both the original and shuffled test sets. The results are also shown in Table 3.

Table 3: Accuracy on original and shuffled data for all configurations

Configuration	Original	Seed 42	Seed 123	Seed 456	Seed 789
CE + Dist	98.87 ± 5.33	98.87 ± 5.33	98.87 ± 5.33	98.87 ± 5.33	98.87 ± 5.33
MSE + Dist	99.03 ± 5.22	99.03 ± 5.22	99.03 ± 5.22	99.03 ± 5.22	99.03 ± 5.22
CE + SVD	99.33 ± 4.62	99.33 ± 4.62	99.33 ± 4.62	99.33 ± 4.62	99.33 ± 4.62
MSE + SVD	98.96 ± 5.46	98.96 ± 5.46	98.96 ± 5.46	98.96 ± 5.46	98.96 ± 5.46
MLP	62.24 ± 14.73	78.76 ± 13.02	77.41 ± 12.97	77.61 ± 12.97	76.05 ± 13.68

As can be seen from the table, the CNN performs well even on the shuffled test set, maintaining high accuracy. This indicates locality: the model relies on the local structure within 2×2 blocks rather than on their global arrangement. In contrast, the MLP yields notably lower accuracy on both the original and shuffled test sets compared to the CNN.

These results confirm the local structure of the generated data, which aligns with the known inductive bias of CNNs.

Now we slightly modify the CNN architecture and conduct an experiment to test translation invariance.

The CNN architecture is as follows: a convolution with a 2×2 kernel and stride 1, followed by ReLU activation and 2×2 max pooling, resulting in 4 feature maps of size 1×1. After flattening, a fully connected layer maps the 4 features to the output classes.

The obtained dataset is presented on Figure 10.

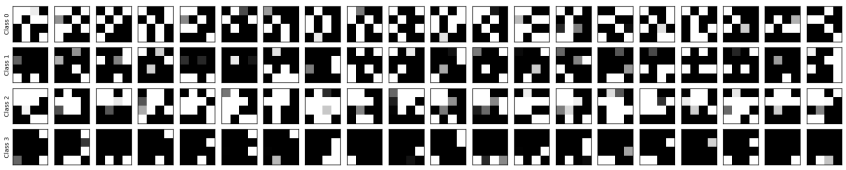


Figure 10: Obtained dataset

To evaluate translation invariance, we train the target model on the original training set and measure its accuracy on the original test set and on test sets shifted by 1 pixel horizontally, vertically, and in both directions. The results are averaged over 1000 random train-test splits and summarized in Table 4.

Table 4: Accuracy on original and shifted images

Shift type	Accuracy (%)
Original	98.78 ± 8.82
Shift horizontal (1px)	95.23 ± 10.12
Shift vertical (1px)	94.56 ± 11.34
Shift horizontal+vertical (1px)	89.12 ± 14.56

As can be observed, the accuracy on the shifted datasets exhibits only a minor drop, indicating translation invariance. This aligns with the well-known inductive bias of CNNs.

Thus, our method demonstrates strong performance on CNNs.

4.3 RNNs

Now we conduct an experiment with RNNs.

The architecture of the target model is as follows: the RNN consists of a single recurrent layer with a hidden dimension of 16. It takes sequences of real-valued numbers as input and produces a prediction for each time step via a linear layer applied to the hidden states.

The experimental parameters are as follows: a sequence of 5000 elements, each of which is a real number. The inner loop has 5 steps, with learning rates of 0.1 and 0.01 for the inner and outer loops, respectively. In this experiment, we avoid the trivial case of a constant sequence by simultaneously training two models in the inner loop: the target RNN and an MLP. The meta-loss is defined as:

$$\mathcal{L} = \text{MSE}_{\text{RNN}} - \alpha \ln(\text{MSE}_{\text{MLP}}) + \lambda \mathcal{R}_{L1},$$

where MSE_{RNN} and MSE_{MLP} are the mean squared errors of the RNN and MLP, respectively, and $\mathcal{R}_{L1}$ is the L1 regularization term. In our experiment, $\alpha = 1$ and $\lambda = 1 \times 10^{-5}$.

Unlike the CNN experiment, the task is formulated as next-element prediction. At each meta-training step, a dataset is constructed from the original sequence using a sliding window, where each example consists of a context of length 10 and the target value corresponding to the element immediately following the context.

As a result of meta-training, we obtained a sequence for which the RNN achieves an MSE of 1.13 ± 0.04, while the MLP yields an MSE of $(4.9 \pm 150) \cdot 10^{27}$. Figure 11 shows an example of RNN predictions on the test set.

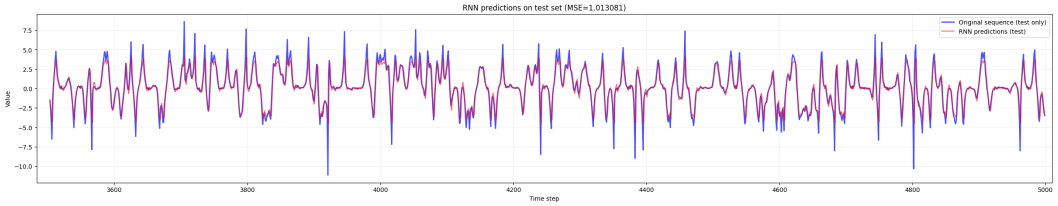


Figure 11: Comparison of the obtained test set and RNN predictions.

To verify the sequential structure of the obtained dataset, we perform the following experiment. First, we train the model on the original training and than evaluate it on two different test set: original and randomly permuted (within each context, the 10 numbers are randomly permuted) test set. Then we train and test model on randomly permuted data. The results are averaged over multiple random model initializations and presented in Table 5.

Table 5: MSE comparison on original and shuffled data

Training	**Test**	**MSE**
Original	Original	1.13 ± 0.04
Original	Shuffled	7.21 ± 0.26
Shuffled	Shuffled	5.973 ± 0.004

The substantial increase in MSE (from 1.13 to 7.21) when testing on randomly shuffled data demonstrates that the model heavily relies on the order of elements. This confirms that the generated sequence possesses a strong temporal structure.

We also examine the dependence of the squared error on the context length. To this end, for each context length, we randomly sample 1500 examples of this length from the test set and compute the corresponding squared prediction error of the RNN. The results, including the mean, median, and interquartile range, are shown in the Figure 12.

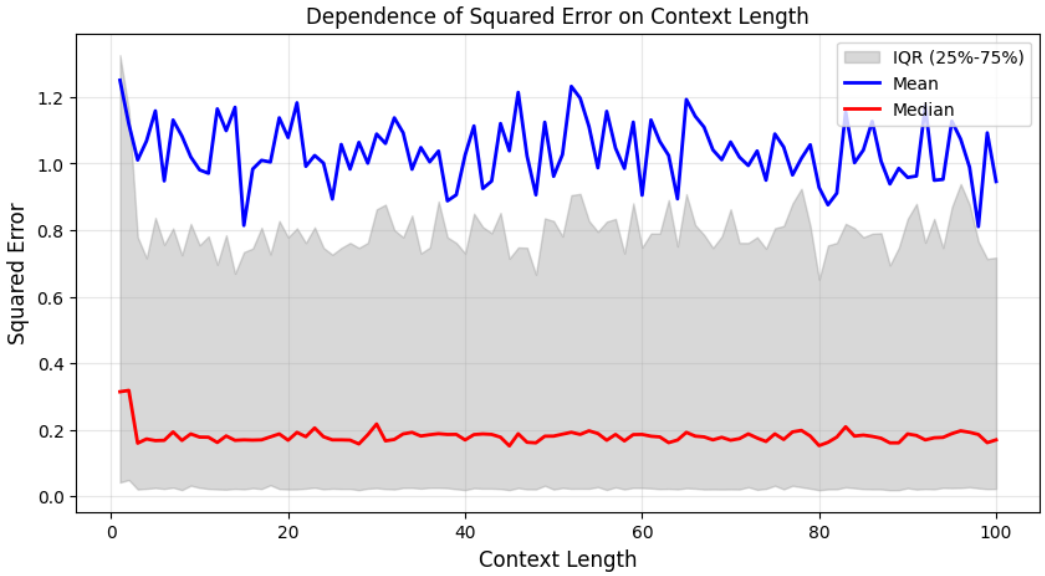


Figure 12: Dependence of Squared Error on context length.

The initial decrease in median squared error up to a context length of 3 indicates that the model benefits from additional preceding elements. The subsequent saturation suggests that the relevant dependencies in the sequence are short-range and that the RNN successfully captures them within a small window. This behavior is characteristic of sequential data with limited memory.

These results collectively confirm that the meta-learned generator produces sequences with a strong sequential structure, which aligns with the inductive bias of RNNs. Therefore, our method demonstrates strong performance on RNNs.

We additionally investigate whether the synthetic data generated for the recurrent learner exhibits a fading-memory structure. The generator is a directly trainable tensor containing 24 real-valued sequences of length 300. Each sequence is converted into overlapping next-value prediction examples with context length 30. The target learner is a vanilla RNN with a single tanh recurrent layer of hidden dimension 16 and a linear output layer. At each meta-training iteration, complete sequences are randomly split into training and test groups in the proportion $0.75/0.25$, and 128 context–target windows are sampled from each group. Three independently initialized RNNs are trained on the same windows for five inner-loop SGD steps with learning rate 0.1 using MSE. The outer loss is the target-variance normalized MSE,

$$\mathcal{L}_{\mathrm{NMSE}} = \frac{\frac{1}{n}\sum_{i=1}^{n}\left(f_\theta(x_i) - y_i\right)^2}{\mathrm{Var}(y) + \varepsilon}, \qquad \varepsilon = 10^{-6}.$$

The generator is optimized for 3000 Adam steps with learning rate 10^{-2}. To prevent degenerate sequences, we use two regularizers. A variance floor prevents complete sequences from collapsing to constants,

$$\mathcal{R}_{\mathrm{var}} = \lambda_{\mathrm{var}} \frac{1}{S}\sum_{s=1}^{S}\left[v_{\min} - \mathrm{Var}(X_s)\right]_+^2,$$

and a temporal-difference floor requires a minimum amount of local variation,

$$\mathcal{R}_{\mathrm{diff}} = \lambda_{\mathrm{diff}} \frac{1}{S}\sum_{s=1}^{S}\left[d_{\min} - \frac{1}{T-1}\sum_{t=1}^{T-1}(x_{s,t+1} - x_{s,t})^2\right]_+^2,$$

with $v_{\min} = 0.05$, $d_{\min} = 0.01$, and $\lambda_{\mathrm{var}} = \lambda_{\mathrm{diff}} = 0.2$. The complete meta-objective is therefore

$$\mathcal{L}_{\mathrm{meta}} = \frac{1}{K}\sum_{k=1}^{K}\mathcal{L}_{\mathrm{NMSE}}^{(k)} + \mathcal{R}_{\mathrm{var}} + \mathcal{R}_{\mathrm{diff}}, \qquad K = 3.$$

Figure 13 shows examples of the generated sequences together with one-step predictions of a recurrent model trained on the frozen dataset.

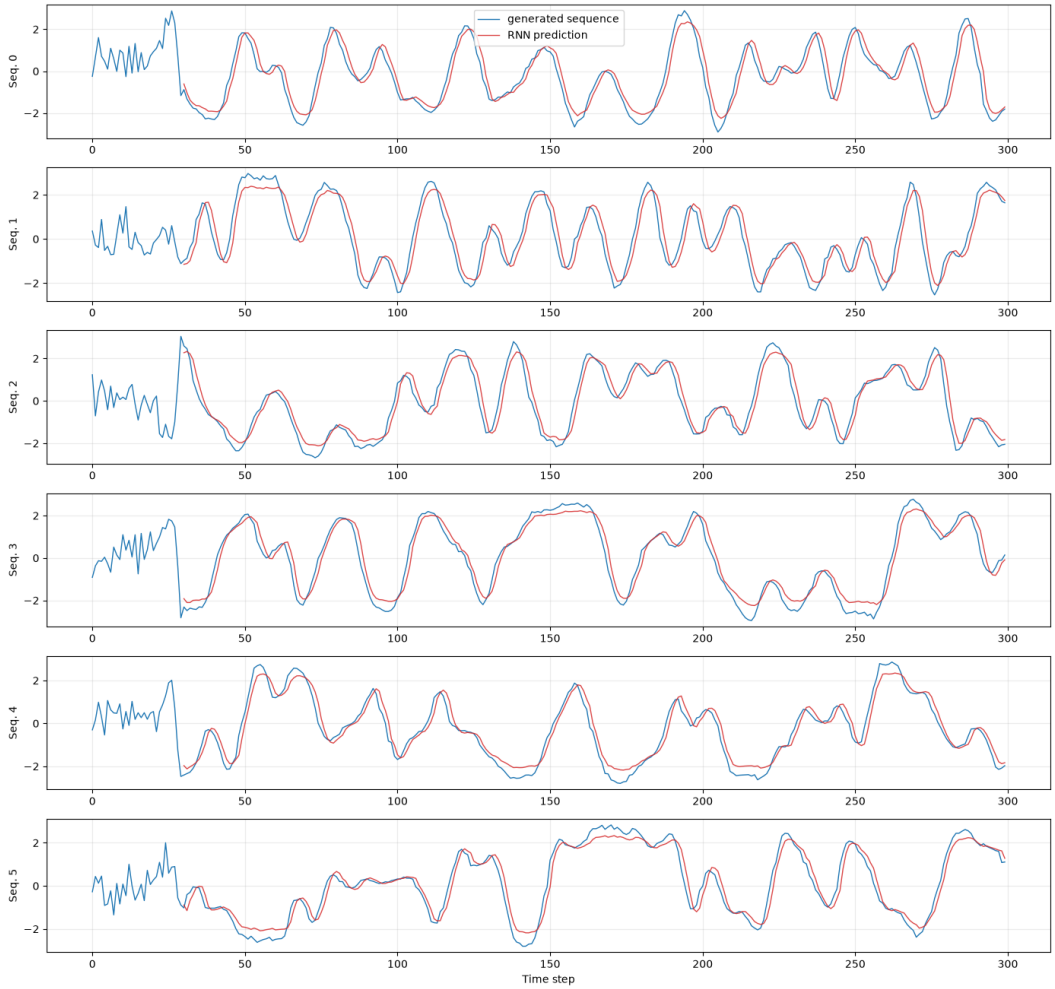


Figure 13: Examples of the generated sequences and one-step predictions of a vanilla RNN. Predictions are produced from contexts of length 30.

To determine how the influence of a context element depends on its temporal distance from the target, we perform two perturbation tests for lags $k \in \{1, \ldots, 30\}$, where $k = 1$ denotes the most recent context element. In each test, we modify only x_{t-k} and leave all other elements unchanged. In the first test, the selected value is replaced by the value at the same position from another randomly chosen test context; in the second test, Gaussian noise with standard deviation 0.5 is added to the selected value. For every lag, we measure both the absolute prediction change and the increase in normalized prediction error. The audit is repeated over 100 random train–test splits and ten independent perturbations per lag. To distinguish a property of the generated data from a property specific to recurrent computation, we repeat the same procedure with a feed-forward MLP that receives the flattened context and has two tanh hidden layers of width 32, i.e. no recurrent state or explicit temporal architecture. As shown in Figure 14, both the RNN and the MLP become less sensitive to a context element as its distance from the predicted value increases.

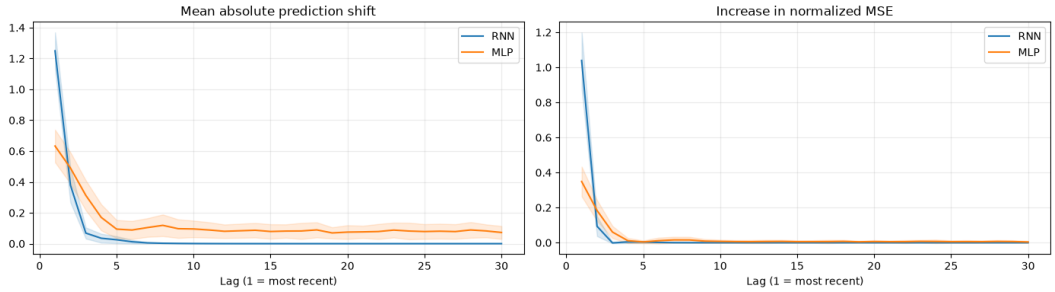


Figure 14: Fading-memory audit for the RNN and the window MLP. The upper row corresponds to replacing one context element with a donor value, and the lower row corresponds to adding Gaussian noise. The columns show the absolute prediction change and the increase in normalized mean squared error. Lag 1 is the most recent element. Curves show the mean over random splits; shaded regions show one standard deviation.

The similarity between the two models indicates that the observed decay is not caused solely by the recurrent architecture; instead, the meta-generated dataset itself contains predominantly short-range predictive dependencies. Finally, we compute a model-independent statistic: the absolute Pearson correlation between the target x_t and the context element x_{t-k}. Its decay with increasing lag, shown in Figure 15, provides additional evidence that the fading-memory pattern is encoded in the generated sequences rather than being only an artifact of the trained predictor.



Figure 15: Absolute Pearson correlation between a context element and the target as a function of temporal lag. Lag 1 corresponds to the context element immediately preceding the target.

4.4 Deep Sets

We consider a classification problem in which each object is a set of ten points in $\mathbb{R}^2$. The dataset is represented by a trainable tensor, while the class labels remain fixed. The point coordinates are bounded to $[-1, 1]$ using the hyperbolic tangent.

The target model is a Deep Sets classifier. First, the same two-layer MLP is applied independently to every point. The resulting representations are averaged and passed to another two-layer MLP that predicts the class. Due to the symmetric aggregation operation, the model output is invariant to the order of points in a set.

We conduct experiments with two and four classes. The two-class dataset contains 100 sets per class, and the hidden and embedding dimensions of the classifier are equal to 32. The four-class dataset contains 20 sets per class, and both dimensions are increased to 128.

At every meta-training iteration, the dataset is divided into stratified training and test subsets in the proportion $0.8/0.2$. The meta-loss is averaged over five independently initialized learners. Each learner is trained for seven inner-loop steps with learning rate 0.5. The generator is optimized for 2000 iterations using Adam with learning rate $3 \cdot 10^{-3}$.

The standard cross-entropy loss is used in the inner loop. In the outer loop, the loss is calculated separately for every pair of classes and combined using a smooth maximum. This prevents the generator from producing a dataset in which only some of the classes are well separated.

We additionally penalize points that are too close within the same set:

$$\mathcal{R}(\mathfrak{D}_{\text{syn}}) = \frac{\lambda}{N\binom{10}{2}} \sum_{i=1}^{N} \sum_{j<k} \left[d_{\min} - \|x_{ij} - x_{ik}\|_2 \right]_+^2 .$$

We set $\lambda = 100$ and use $d_{\min} = 0.4$ for two classes and $d_{\min} = 0.2$ for four classes.

The generated datasets are presented in Figures 16 and 17. Each subplot represents one set, while the rows correspond to different classes. The color gradient indicates the stored index of a point and is used only for visualization.

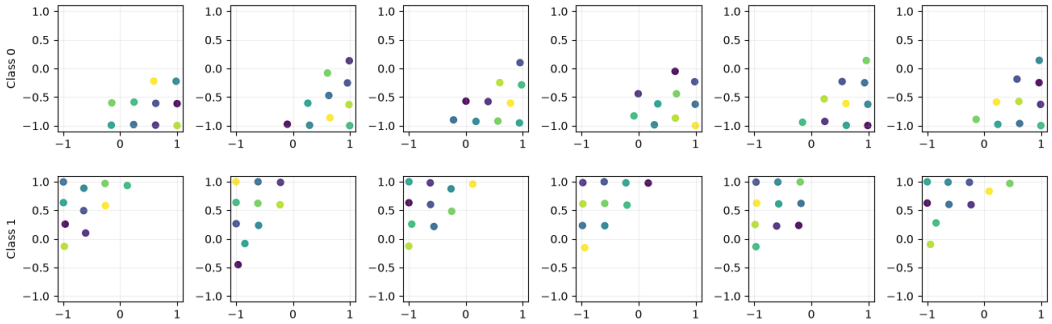


Figure 16: Examples of sets from the generated two-class dataset.

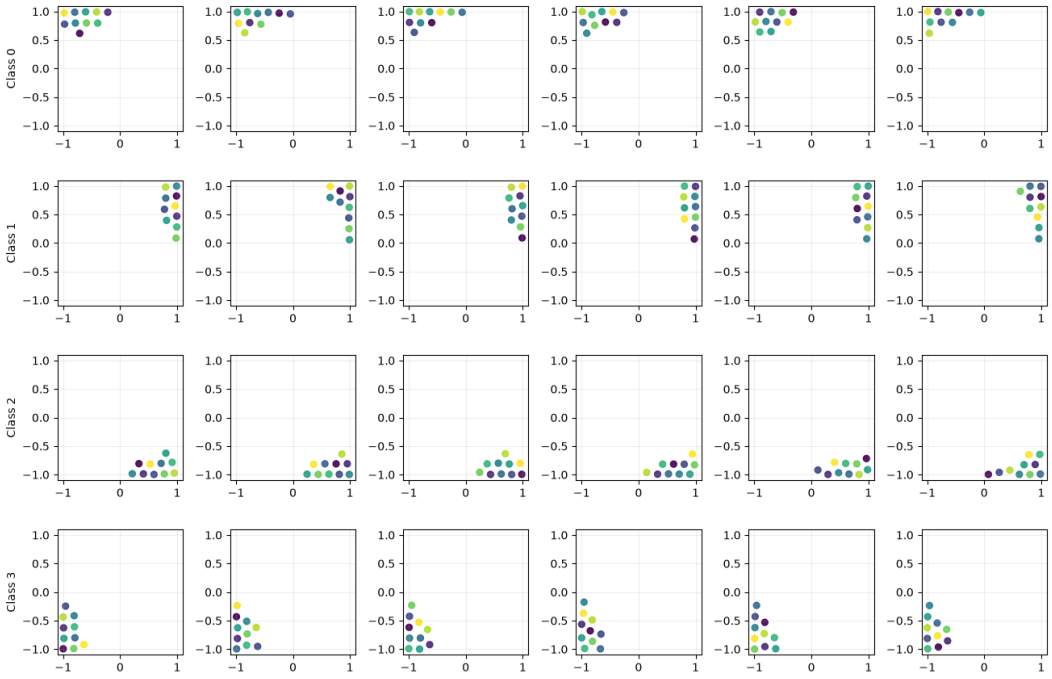


Figure 17: Examples of sets from the generated four-class dataset.

To evaluate permutation invariance, we compare Deep Sets with an ordered MLP. The MLP receives all point coordinates as one flattened vector and therefore has no built-in permutation invariance. Both models are trained either on the original training sets or on sets whose points are randomly permuted. After training, they are evaluated on both original and permuted test data.

The evaluation is repeated for 100 random train–test splits and 50 independent permutations per split. In addition to accuracy, we measure the fraction of changed predictions, changes in logits and probabilities, and the correct-class margin. The results are reported in Tables 6 and 7.

Table 6: Permutation audit for the two-class dataset.

Model	Training data	Acc. orig.	Acc. perm.	Changed pred.	Mean $\lvert\Delta z\rvert$	Max $\lvert\Delta z\rvert$	Mean $\lVert\Delta p\rVert_1$	Margin orig.	Margin perm.
Deep Sets	Original	1.0000	1.0000	0.0000	0.0000	0.0000	0.0000	1.2460	1.2460
Deep Sets	Permuted	1.0000	1.0000	0.0000	0.0000	0.0000	0.0000	1.1867	1.1867
Ordered MLP	Original	1.0000	1.0000	0.0000	0.1244	0.9605	0.0222	3.0030	2.9953
Ordered MLP	Permuted	1.0000	1.0000	0.0000	0.1265	0.8872	0.0225	3.0104	3.0169

Table 7: Permutation audit for the four-class dataset.

Model	Training data	Acc. orig.	Acc. perm.	Changed pred.	Mean $\lvert\Delta z\rvert$	Max $\lvert\Delta z\rvert$	Mean $\lVert\Delta p\rVert_1$	Margin orig.	Margin perm.
Deep Sets	Original	1.0000	1.0000	0.0000	0.0000	0.0000	0.0000	2.1620	2.1620
Deep Sets	Permuted	1.0000	1.0000	0.0000	0.0000	0.0000	0.0000	2.1253	2.1253
Ordered MLP	Original	1.0000	1.0000	0.0000	0.0732	0.5644	0.0134	3.5091	3.5115
Ordered MLP	Permuted	1.0000	1.0000	0.0000	0.0736	0.5224	0.0137	3.4861	3.4909

Both models achieve perfect accuracy on the original and permuted test sets. However, this result has a different interpretation for the two architectures. For Deep Sets, permutations leave the logits, probabilities, and predicted classes unchanged. Thus, the expected architectural invariance is preserved exactly.

The ordered MLP also retains perfect accuracy, but its logits and predicted probabilities change after permutation. Therefore, the MLP has not learned exact permutation invariance. Instead, the generated classes are sufficiently well separated for the permutation-induced changes not to cross the decision boundary. This is also supported by the large positive correct-class margins, which remain almost unchanged after permutation.

Consequently, classification accuracy alone is not sufficient to test permutation invariance. The experiment shows that the generated dataset is consistent with the inductive bias of Deep Sets: an invariant model can be trained successfully and generalizes to unseen sets. At the same time, the geometric separation of the classes makes the task simple enough for an order-sensitive model to remain accurate under permutations. The difference between the models becomes visible only when their complete outputs, rather than predicted labels alone, are compared.

5 Conclusion

We propose a generative meta-learning framework for extracting the inductive bias of differentiable learning algorithms without prior knowledge of the input data distribution. Unlike prior work [12] that generates only labels for samples from a fixed distribution, our approach produces entire synthetic datasets via a meta-learned generative model. The resulting datasets are easy for the target model to generalize from and reflect its inductive bias.

We empirically validate our method in experiments with CNNs and RNNs and extract the well-known inductive biases of these architectures: locality and translation invariance for CNNs, and sequential structure for RNNs.

Thus, our method successfully extracts inductive bias, offering a tool for analyzing different model architectures.

References

[1] Tom M Mitchell. The need for biases in learning generalizations. 1980.

[2] Foster John Provost and Bruce G Buchanan. Inductive policy: The pragmatics of bias selection. *Machine Learning*, 20(1):35–61, 1995.

[3] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791.

[4] Zihao Wang and Lei Wu. Theoretical analysis of inductive biases in deep convolutional networks, 2024. URL https://arxiv.org/abs/2305.08404.

[5] Nadav Cohen and Amnon Shashua. Inductive bias of deep convolutional networks through pooling geometry, 2017. URL https://arxiv.org/abs/1605.06743.

[6] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Comput.*, 9 (8):1735–1780, November 1997. ISSN 0899-7667. doi: 10.1162/neco.1997.9.8.1735. URL https://doi.org/10.1162/neco.1997.9.8.1735.

[7] Taiga Ishii, Ryo Ueda, and Yusuke Miyao. Empirical analysis of the inductive bias of recurrent neural networks by discrete fourier transform of output sequences, 2023. URL https://arxiv.org/abs/2305.09178.

[8] Igor Dubinin and Felix Effenberger. Fading memory as inductive bias in residual recurrent networks. *Neural Networks*, 173:106179, 2024. ISSN 0893-6080. doi: https://doi.org/10.1016/j.neunet.2024.106179. URL https://www.sciencedirect.com/science/article/pii/S0893608024001035.

[9] Akhilan Boopathy, William Yue, Jaedong Hwang, Abhiram Iyer, and Ila Fiete. Towards exact computation of inductive bias, 2024. URL https://arxiv.org/abs/2406.15941.

[10] Chenglei Si, Dan Friedman, Nitish Joshi, Shi Feng, Danqi Chen, and He He. Measuring inductive biases of in-context learning with underspecified demonstrations, 2023. URL https://arxiv.org/abs/2305.13299.

[11] Michael Y Li, Erin Grant, and Thomas L Griffiths. Meta-learning inductive biases of learning systems with gaussian processes. In *Fifth Workshop on Meta-Learning at the Conference on Neural Information Processing Systems*, 2021.

[12] William Dorrell, Maria Yuffa, and Peter Latham. Meta-learning the inductive biases of simple neural circuits, 2023. URL https://arxiv.org/abs/2211.13544.

[13] Sébastien M. R. Arnold, Praateek Mahajan, Debajyoti Datta, Ian Bunner, and Konstantinos Saitas Zarkias. learn2learn: A library for meta-learning research, 2020. URL https://arxiv.org/abs/2008.12284.

[14] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks, 2017. URL https://arxiv.org/abs/1703.03400.

[15] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017. URL https://arxiv.org/abs/1412.6980.